Travel Companion – AI Restaurant Finder

Varianta Secvențială (S6)

Algoritmi Paraleli și Distribuiți

Stoian Cristian Georgian
Calculatoare Română, Grupa CR3.2A

Martie 2026

Cuprins

1 Cerințe și Tema Proiectului **3**
1.1 Descrierea Proiectului . . . 3
1.2 Obiectivul Principal . . . 3
1.3 Modele de Execuție . . . 3
1.4 Tehnologii Utilizate . . . 3
1.5 Surse de Date API . . . 3
1.6 Pipeline de Execuție . . . 4

2 Arhitectura Sistemului **4**
2.1 Strategy Pattern . . . 4
2.2 Structura Proiectului . . . 4
2.3 Modelul de Date . . . 5
2.4 Formula de Scoring . . . 6

3 Specificații Mașină **6**

4 Implementare Secvențială **6**
4.1 Descrierea Fluxului . . . 6
4.2 Tratarea Erorilor . . . 7
4.3 Instrumentare Timing . . . 7

5 Rezultate Experimentale **8**
5.1 Configurarea Experimentelor . . . 8
5.2 Rezultate – Timpi de Execuție . . . 8
5.3 Timpi Medii per Sursă API . . . 8
5.4 Analiza Rezultatelor . . . 9
5.4.1 Dominanța latenței API . . . 9
5.4.2 Variabilitatea Overpass API . . . 9
5.4.3 Scalabilitate Liniară . . . 9
5.4.4 Oportunitate de Paralelizare . . . 9

6 Utilizare **10**
6.1 Interfața CLI 10
6.2 Parametri CLI 10

7 Concluzii și Direcții Viitoare **10**
7.1 Concluzii S6 10
7.2 Direcții Viitoare 11

1 Cerințe și Tema Proiectului

1.1 Descrierea Proiectului

Travel Companion este o aplicație bazată pe inteligență artificială care recomandă cele mai bune restaurante pe baza preferințelor utilizatorului (locație, tip de mâncare, mediu, distanță etc.).

Sistemul agregă date de la multiple surse API externe și aplică algoritmi de scoring cu ponderi configurabile pentru a determina cele mai potrivite restaurante.

1.2 Obiectivul Principal

Obiectivul principal al proiectului este **analiza și compararea abordărilor secvențiale și paralele** pentru agregarea și procesarea datelor din surse multiple. Această livrare (S6) prezintă implementarea și analiza variantei secvențiale.

1.3 Modele de Execuție

1. **Secvențial** (S6 – această livrare): Toate operațiile se execută una după alta
2. **Paralel #1 – Async I/O**: Apeluri API concurente folosind `asyncio`
3. **Paralel #2 – Multiprocessing**: Procesare CPU paralelă pentru scoring
4. **Paralel #3 – Hibrid**: Async I/O + Multiprocessing combinat

1.4 Tehnologii Utilizate

- **Limbaj**: Python 3.12.3
- **Pattern arhitectural**: Strategy Pattern (interfață comună pentru toate modurile de execuție)
- **Surse de date**: Clienți API reali (Geoapify, Overpass/OSM, Foursquare)
- **Instrumentare**: `TimingCollector` cu decorator `@timed_task` pentru măsurarea timpilor
- **Dependențe**: `requests`, `python-dotenv`

1.5 Surse de Date API

Sistemul utilizează trei surse de date reale pentru restaurante:

1. **Geoapify Places API** – oferă date despre restaurante incluzând rating (bazat pe popularitate), locație, categorii, nivel de preț și adresă formatată. Suportă filtrarea pe categorii specifice (e.g. `catering.restaurant.seafood`) și filtre geografice circulare.

2. **Overpass API (OpenStreetMap)** – furnizează date deschise din baza de date OpenStreetMap. Permite interogări complexe prin limbajul Overpass QL pentru restaurante cu filtrare pe tipul de bucătărie (`cuisine`). Extrage tag-uri de mediu (terasă, loc de joacă, seating exterior) și adrese din componentele OSM. Nu necesită cheie API.

3. **Foursquare Places API** – adaugă o a treia perspectivă cu căutare text-based și categorii detaliate. Autentificarea se face prin Bearer token.

Toți clienții implementează aceeași interfață abstractă `BaseAPIClient`, permițând adăugarea sau înlocuirea transparentă a oricărei surse de date.

1.6 Pipeline de Execuție

Fluxul secvențial al pipeline-ului:

$$T_{total} = T_{API_1} + T_{API_2} + \ldots + T_{API_N} + T_{dedup} + T_{scoring} + T_{ranking}$$

Unde fiecare T_{API_i} include latența de rețea și timpul de procesare a răspunsului, iar T_{dedup}, $T_{scoring}$, $T_{ranking}$ sunt timpii de procesare locală.

2 Arhitectura Sistemului

2.1 Strategy Pattern

Arhitectura se bazează pe **Strategy Pattern**, unde interfața abstractă `ExecutionStrategy` definește metoda `execute()` implementată de fiecare mod de execuție:

Listing 1: Interfața ExecutionStrategy

```python
class ExecutionStrategy(ABC):
    def __init__(self, collector: TimingCollector):
        self.collector = collector

    @abstractmethod
    def execute(self, query, clients, scorer):
        pass

    @property
    @abstractmethod
    def mode_name(self) -> str:
        pass
```

Pentru S6, este implementată doar `SequentialStrategy`. Variantele paralele se vor adăuga fără modificarea codului existent, datorită polimorfismului oferit de interfața comună.

2.2 Structura Proiectului

- `models.py` – Clase de date: `Query`, `Restaurant`, `ScoredRestaurant`, `PipelineResult`
- `config.py` – Configurație: chei API, ponderi scoring, valori implicite

- `timing.py` – `TimingCollector`, `TimingRecord` și decoratorul `@timed_task`
- `api_clients/` – Clienți API:
  - `base.py` – Interfața abstractă `BaseAPIClient`
  - `real_client.py` – Implementări reale: `GeoapifyClient`, `OverpassClient`, `FoursquareClient`
  - `mock_client.py` – Clienți mock pentru teste (date sintetice cu latență simulată)
- `execution/` – Strategii de execuție:
  - `base.py` – Interfața abstractă `ExecutionStrategy`
  - `sequential.py` – Implementarea secvențială
- `scoring.py` – Formula de scoring cu ponderi configurabile și deduplicare
- `pipeline.py` – Orchestratorul pipeline-ului
- `main.py` – Punct de intrare CLI
- `experiments.py` – Runner pentru experimente batch cu export CSV

2.3 Modelul de Date

Listing 2: Clasele principale de date

```python
@dataclass
class Query:
    location: str            # "Constanta"
    food_types: List[str]    # ["seafood", "fish"]
    environment: List[str]   # ["seaside", "terrace"]
    max_distance_km: float   # 10.0
    latitude: Optional[float] = None
    longitude: Optional[float] = None

@dataclass
class Restaurant:
    name: str
    source: str              # "geoapify", "overpass_osm", "foursquare"
    rating: float            # 0-5
    latitude: float
    longitude: float
    distance_km: float
    categories: List[str]
    price_level: int         # 1-4
    environment_tags: List[str]
    review_count: int
    address: str
```

2.4 Formula de Scoring

$$score = 0.30 \cdot S_{rating} + 0.25 \cdot S_{distance} + 0.30 \cdot S_{food} + 0.15 \cdot S_{environment}$$

Unde:

- $S_{rating} = \frac{rating}{5.0}$ – normalizare la intervalul $[0, 1]$
- $S_{distance} = \max(0, 1 - \frac{d}{d_{max}})$ – penalizare liniară cu distanța
- S_{food} = raportul de potrivire fuzzy între tipurile de mâncare căutate și categoriile restaurantului (matching pe subșiruri)
- $S_{environment}$ = raportul de potrivire fuzzy între preferințele de mediu și tag-urile restaurantului

Deduplicare: Restaurantele cu același nume aflate la mai puțin de 100m distanță sunt considerate duplicate. Se păstrează versiunea cu rating-ul mai mare.

3 Specificații Mașină

Parametru	Valoare
CPU	AMD Ryzen 7 7745HX with Radeon Graphics
Nuclee fizice	8
Nuclee logice	16 (8 cores × 2 threads)
RAM	16 GB
Sistem de operare	Linux 6.6.87.2-microsoft-standard-WSL2 x86_64
Python	3.12.3
Conexiune rețea	Wi-Fi (apeluri API reale către servere externe)

Tabela 1: Specificațiile mașinii de test

4 Implementare Secvențială

4.1 Descrierea Fluxului

În modul secvențial, toate operațiile se execută strict una după alta:

1. Se construiește query-ul din parametrii utilizatorului (locație, tip mâncare, mediu)
2. Se apelează API #1 (**Geoapify**) – se așteaptă răspunsul complet
3. Se apelează API #2 (**Overpass/OSM**) – se așteaptă răspunsul complet
4. Se apelează API #3 (**Foursquare**) – se așteaptă răspunsul complet (opțional)
5. Se deduplică rezultatele (eliminare duplicate pe bază de nume + locație)
6. Se calculează scorurile pentru fiecare restaurant
7. Se sortează restaurantele descrescător după scor

8. Se returnează top K rezultate

Listing 3: Implementarea SequentialStrategy

```
class SequentialStrategy(ExecutionStrategy):
    mode_name = "sequential"

    def execute(self, query, clients, scorer):
        all_restaurants = []
        for client in clients:
            start = time.perf_counter()
            try:
                results = client.search(query)
            except Exception as e:
                # Log error, continue with other APIs
                continue
            end = time.perf_counter()
            self.collector.record(
                f"api_{client.name}", start, end,
                count=len(results))
            all_restaurants.extend(results)

        # Deduplication, scoring, ranking
        deduped = scorer.deduplicate(all_restaurants)
        scored = scorer.score_all(deduped, query)
        ranked = sorted(scored,
                        key=lambda x: x.score,
                        reverse=True)
        return ranked
```

4.2 Tratarea Erorilor

Dacă un client API eșuează (timeout, eroare de rețea, răspuns invalid), execuția continuă cu ceilalți clienți. Eroarea este înregistrată în `TimingCollector` cu metadatele corespunzătoare, iar utilizatorul primește un avertisment.

4.3 Instrumentare Timing

Fiecare etapă este instrumentată cu `TimingCollector`, care înregistrează:

- Timpul de start și end (relativ la începutul pipeline-ului), folosind `time.perf_counter()`
- Durata în secunde
- Metadate: număr de rezultate returnate, mesaje de eroare (dacă există)

Datele de timing sunt exportate în format CSV prin metoda `to_csv_dict()`, care aplatizează înregistrările într-un dicționar cu coloane separate per etapă de pipeline.

5 Rezultate Experimentale

5.1 Configurarea Experimentelor

Experimentele au fost rulate folosind **clienți API reali** (Geoapify, Overpass, Foursquare), cu apeluri de rețea efective către serverele externe.

- **6 query-uri diferite** (orașe și tipuri de mâncare variate):
  - Constanța / seafood / seaside
  - București / italian + pizza / cozy
  - Cluj-Napoca / romanian + traditional
  - Timișoara / fast food / central
  - Iași / french / elegant
  - Brașov / steakhouse / rustic + terrace
- 1, 2 și 3 surse API per configurație
- 3 rulări per query per configurație
- Total: $6 \times 3 \times 3 = 54$ rulări

5.2 Rezultate – Timpi de Execuție

Nr. APIs	**Media (s)**	**Std Dev (s)**	**Min / Max (s)**
1 API (Geoapify)	0.6907	0.2442	0.4190 / 1.2432
2 APIs (Geoapify + Overpass)	9.0550	3.5290	1.4212 / 14.1986
3 APIs (+ Foursquare)	8.7294	4.5106	1.7785 / 16.7381

Tabela 2: Timpi de execuție totali – mod secvențial

5.3 Timpi Medii per Sursă API

Sursă API	**Timp mediu (s)**	**Observații**
Geoapify	$\sim$0.69 – 0.78	Rapid, răspunsuri consistente
Overpass (OSM)	$\sim$7.39 – 8.35	Foarte variabil, depinde de încărcarea serverului
Foursquare	$\sim$0.56	Rapid, răspunsuri consistente
Deduplicare	< 0.001	Neglijabil
Scoring	< 0.001	Neglijabil
Ranking	< 0.001	Neglijabil

Tabela 3: Timpi medii per componentă a pipeline-ului

5.4 Analiza Rezultatelor

5.4.1 Dominanța latenței API

Datele experimentale confirmă că **latența API domină complet timpul de execuție**. Timpii de procesare locală (deduplicare, scoring, ranking) sunt sub 1ms, reprezentând mai puțin de 0.01% din timpul total.

$$T_{total} \approx \sum_{i=1}^{N} T_{API_i}$$

5.4.2 Variabilitatea Overpass API

API-ul Overpass (OpenStreetMap) prezintă cea mai mare variabilitate, cu timpi între 0.47s și 15.27s. Aceasta se datorează:

- Încărcării serverelor publice Overpass (shared infrastructure)
- Complexității interogărilor Overpass QL (filtrare pe rază + tip bucătărie)
- Lipsei unui SLA garantat (serviciu gratuit, open-source)

5.4.3 Scalabilitate Liniară

Cu 1 API, timpul mediu este ∼0.69s. Cu 2 APIs, crește la ∼9.05s (dominat de Overpass). Cu 3 APIs, timpul rămâne similar (∼8.73s) deoarece Foursquare este rapid (∼0.56s) și se adaugă la un total deja dominat de Overpass.

Comportamentul este strict secvențial:

$$T_{2\text{ APIs}} \approx T_{\text{Geoapify}} + T_{\text{Overpass}} \approx 0.71 + 8.35 = 9.06s$$

$$T_{3\text{ APIs}} \approx T_{\text{Geoapify}} + T_{\text{Overpass}} + T_{\text{Foursquare}} \approx 0.78 + 7.39 + 0.56 = 8.73s$$

5.4.4 Oportunitate de Paralelizare

Apelurile API sunt **complet independente** – nu există dependențe de date între ele. În modul secvențial, fiecare apel blochează execuția până la primirea răspunsului. Cu paralelizare (async I/O), timpul ar scădea la:

$$T_{paralel} \approx \max(T_{API_1}, T_{API_2}, T_{API_3})$$

Speedup-ul teoretic pentru 3 APIs:

$$S = \frac{T_{secvential}}{T_{paralel}} = \frac{T_1 + T_2 + T_3}{\max(T_1, T_2, T_3)} \approx \frac{8.73}{7.39} \approx 1.18\times$$

În cazul acesta, speedup-ul este modest deoarece Overpass domină. Pentru API-uri cu latențe echilibrate, speedup-ul ar fi mai aproape de N (numărul de surse).

6 Utilizare

6.1 Interfața CLI

Listing 4: Exemple de utilizare

```
# Instalare dependente
pip install -r requirements.txt

# Cautare restaurante in Constanta
python main.py --location "Constanta" \
    --food "seafood,fish" \
    --environment "seaside,terrace" \
    --apis 2 --top-k 5

# Rulare experimente batch
python experiments.py --runs-per-query 5 \
    --api-counts "1,2,3" \
    --output report/experiment_results.csv
```

6.2 Parametri CLI

Parametru	Implicit	Descriere
`--location`	(obligatoriu)	Numele orașului
`--food`	`""`	Tipuri de mâncare, separate prin virgulă
`--environment`	`""`	Preferințe de mediu, separate prin virgulă
`--max-distance`	10.0	Distanța maximă în km
`--apis`	2	Număr de surse API (1–3)
`--top-k`	10	Număr de rezultate afișate

Tabela 4: Parametrii liniei de comandă

7 Concluzii și Direcții Viitoare

7.1 Concluzii S6

- Varianta secvențială funcționează corect cu API-uri reale și produce recomandări relevante
- Timpii de execuție sunt dominați de latența API (în special Overpass/OSM)
- Procesarea locală (deduplicare, scoring, ranking) este neglijabilă (< 1ms)
- Variabilitatea mare a timpilor Overpass (0.47s – 15.27s) subliniază importanța toleranței la erori
- Arhitectura Strategy Pattern facilitează adăugarea variantelor paralele fără modificarea codului existent

7.2 Direcții Viitoare

1. **Paralel #1 (Async I/O)**: Toate apelurile API lansate simultan cu `asyncio` + `aiohttp` – $T = \max(T_{API_1}, T_{API_2}, T_{API_3})$
2. **Paralel #2 (Multiprocessing)**: Scoring distribuit pe multiple procese pentru seturi mari de date
3. **Paralel #3 (Hibrid)**: Combinarea async I/O (pentru API) + multiprocessing (pentru scoring)
4. Vizualizare performanță: grafice Gantt pentru timeline-ul execuției, grafice speedup